

Feature: Login to OpenTaps

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be logged in successfully

Examples:

username	password
Ramesh_Kumar	SecurePass123
Priya_S	Chennai2024
Venkat_Reddy	Bangalore#99
Lakshmi_N	Mysore@2025

Scenario Outline: Failed login with invalid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should see an error message

Examples:

username	password
Ramesh_Kumar	WrongPassword
Admin_User	Admin123

| Unknown_User | TestPass |

```
node --input-type=module -e "import { getPrompt, CODE_GENERATOR_TYPES } from
'./src/scripts/prompts.js'; const prompt =
getPrompt('CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS', { domContent:
'<input id="username">', pageUrl: 'https://example.test/login' }); if
(!prompt.includes('https://example.test/login') ; !prompt.includes('<input
id="username">') ; !prompt.includes('Example TypeScript output') ;
CODE_GENERATOR_TYPES.CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS !==
'Cucumber-With-Playwright-TypeScript-Steps') process.exit(1); console.log('Playwright
Cucumber prompt resolved successfully');"
```

/**

* Cucumber with Playwright TypeScript Step Definitions

*/

CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS: `

Instructions:

- Generate BOTH:

1. A Cucumber .feature file.

2. A TypeScript step definition class or module for Playwright.

- [CRITICAL] Do NOT include Page Object code.

- [MANDATORY] Use Playwright with @cucumber/cucumber and chromium from playwright.

- [MANDATORY] Include Before and After hooks for browser, context, and page setup/teardown.

- [MANDATORY] Use explicit Playwright waits such as locator.waitFor or expect assertions — never use waitForTimeout.

- [STRICT] Use Scenario Outline with Examples tables and the realistic credentials shown in the example.

- [STRICT] Map every locator only to elements present in the supplied DOM.

- [MANDATORY] Output temperature is fixed at 0.1 for deterministic generation — given the same DOM and URL input, produce identical step names, locator strategy, and wait conditions on every run.

- Return exactly one Gherkin block followed by exactly one TypeScript block. Do not add explanations or duplicate either block.

Context:

Application: Web page under test at the given URL. Steps must map only to elements present in the DOM.

DOM:

\`\`\`html

\\${domContent}

\`\`\`

URL: \\${pageUrl}

Example feature output:

\`\`\`gherkin

Feature: Login to OpenTaps

Scenario Outline: Successful login with valid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be logged in successfully

Examples:

| username | password |

| Ramesh_Kumar | SecurePass123 |

| Priya_S | Chennai2024 |

| Venkat_Reddy | Bangalore#99 |

| Lakshmi_N | Mysore@2025 |

Scenario Outline: Failed login with invalid credentials

Given I open the login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should see an error message

Examples:

username	password	
Ramesh_Kumar	WrongPassword	
Admin_User	Admin123	
Unknown_User	TestPass	

\\`\\`

Example TypeScript output:

\\`\\`\\`typescript

```
import { After, Before, Given, Then, When } from '@cucumber/cucumber';
```

```
import { chromium, Browser, BrowserContext, Page } from 'playwright';
```

```
let browser: Browser;
```

```
let context: BrowserContext;
```

```
let page: Page;
```

```
Before(async () => {
```

```
  browser = await chromium.launch({ headless: true });
```

```
  context = await browser.newContext();
```

```
  page = await context.newPage();
```

```
});
```

```
After(async () => {  
  await context?.close();  
  await browser?.close();  
});
```

```
Given('I open the login page', async () => {  
  await page.goto(`${pageUrl}`, { waitUntil: 'domcontentloaded' });  
});
```

```
When('I type {string} into the Username field', async (username: string) => {  
  const usernameField = page.locator('#username');  
  await usernameField.waitFor({ state: 'visible' });  
  await usernameField.fill(username);  
});
```

```
When('I type {string} into the Password field', async (password: string) => {  
  const passwordField = page.locator('#password');  
  await passwordField.waitFor({ state: 'visible' });  
  await passwordField.fill(password);  
});
```

```
When('I click the Login button', async () => {  
  const loginButton = page.getByRole('button', { name: 'Login' });  
  await loginButton.waitFor({ state: 'visible' });  
  await loginButton.click();  
});
```

```
Then('I should be logged in successfully', async () => {  
  await page.locator('.success').waitFor({ state: 'visible' });  
});
```

```
Then('I should see an error message', async () => {  
  await page.locator('.error').waitFor({ state: 'visible' });  
});
```

```\`

Persona:

- Generate as: QA Automation Engineer skilled in Cucumber, Playwright, and TypeScript.
- Review as: Automation Lead validating step-definition quality, locator stability, wait strategy, and reproducibility.

Output Format:

- Gherkin in a ```\` gherkin block followed by TypeScript in a ```\` typescript block.
- No text outside the two blocks.

Tone:

- [STRICT] Professional, deterministic, executable, structured.

`,`